

Summary

Functions are a common way of grouping our programs' code together, and they require you to make certain decisions: what to name them, how big to make them, how many parameters they should have, and how many arguments you should pass for those parameters. The `*` and `**` syntax in `def` statements allows functions to receive a varying number of parameters, making them variadic functions.

Although not a functional programming language, Python has many features that functional programming languages use. Functions are first-class objects, meaning you can store them in variables and pass them as arguments to other functions (which are called higher-order functions in this context). Lambda functions offer a short syntax for specifying nameless, anonymous functions as the arguments for higher-order functions. The most common higher-order functions in Python are `map()` and `filter()`, although you can execute the functionality they provide faster with list comprehensions.

The return values of your functions should always be the same data type. You shouldn't use return values as error codes: exceptions are for indicating errors. The `None` value in particular is often mistakenly used as an error code.